

Projet Intégrateur IA : Détection de Deepfakes

Reproductibilité, Implémentation et Analyse du modèle F3Net

Camelia REKIOUA

Télécom Paris

Avec l'encadrement de : Mathieu Fontaine

21 juin 2026

Résumé

Ce rapport présente mon implémentation de l'architecture F^3 -Net, un modèle d'intelligence artificielle spécialisé dans la détection de deepfakes développée dans l'article *Thinking in frequency : Face forgery detection by mining frequency-aware clues* [1]. J'y expose la démarche complète permettant de passer de la donnée brute à la prédiction finale : le pipeline de prétraitement que j'ai développé pour la base de données FaceForensics++, les choix d'intégration logicielle qui m'ont conduit à combiner deux implémentations de référence, et l'analyse mathématique des composants clés du modèle. Ce travail de reproduction permet de comprendre non seulement ce que le modèle fait, mais pourquoi il surpasse les approches spatiales sur des médias fortement compressés.

Table des matières

1	Introduction	3
2	Fondements Théoriques : Penser en Fréquences	3
2.1	Intérêt d'une approche fréquentielle	3
2.2	Architecture Globale du Modèle F^3 -Net	4
2.3	Décomposition Adaptative Globale (FAD)	6
2.4	Statistiques de Fréquences Locales (LFS)	7
2.5	Fusion par Attention Croisée (MixBlock)	8
3	Méthodologie et Pipeline Logiciel	10
3.1	Le Dataset FaceForensics++ (FF++)	10
3.2	Prétraitement : Justification du Crop Conservateur	10
3.3	Architecture Logicielle et Choix Techniques	11
3.4	Remarques sur la Reproductibilité	11
4	Expérimentations et Analyse des Résultats	12
4.1	Phase d'Entraînement	12
4.2	Performances et Analyse Critique	12
5	Conclusion	13

1 Introduction

Les modèles génératifs profonds ont franchi un cap décisif ces dernières années. Il est aujourd’hui possible de générer, manipuler ou échanger des visages sur des vidéos avec un niveau de réalisme tel qu’il trompe l’œil humain. Si ces technologies ouvrent des perspectives fascinantes pour la création de contenu, leur usage malveillant soulève des enjeux majeurs de sécurité et de fraude. Détecter des images générées par ces modèles de manière automatisée et surtout fiable est donc devenu un enjeu technologique important.

Historiquement, la majorité des systèmes de détection de deepfakes se sont concentrés sur le domaine spatial, analysant les valeurs de pixels (espaces RVB ou HSV) pour y détecter des anomalies visuelles. Ces méthodes atteignent cependant leurs limites face aux algorithmes de deepfakes modernes comme NeuralTextures, qui dissimulent presque parfaitement leurs traces visuelles. Plus critique encore, lorsque les médias sont diffusés sur les plateformes numériques, ils subissent de fortes compressions (format H.264). Cette dégradation de la qualité efface les derniers indices spatiaux résiduels, rendant les réseaux de neurones classiques aveugles aux manipulations.

C’est pour contourner cette limite que l’architecture F^3 -Net développée dans (*Thinking in Frequency : Face Forgery Detection by Mining Frequency-aware Clues*) [1] hybride une approche spatiale et fréquentielle. Bien que les indices de falsification disparaissent dans le domaine spatial, ils laissent une signature détectable dans le spectre des fréquences sous forme de distributions d’énergie anormales. Ce modèle exploite deux mécanismes complémentaires : une décomposition globale et adaptative du spectre (FAD), couplée à une analyse statistique locale (LFS), fusionnées par un module d’attention croisée (MixBlock).

Ce projet intégrateur a pour objectif de reproduire ce modèle et d’en éprouver la robustesse d’ingénierie sur la base de données FaceForensics++ en compression sévère (c40). À travers ce rapport, je présenterai les fondements théoriques du modèle, les choix techniques que j’ai effectués pour son implémentation, et les résultats obtenus.

2 Fondements Théoriques : Penser en Fréquences

2.1 Intérêt d’une approche fréquentielle

Les systèmes de détection classiques s’appuient sur des approches spatiales : ils analysent directement les valeurs de pixels pour détecter les empreintes laissées par les algorithmes qui génèrent ces deepfakes. Ces méthodes ont montré des résultats remarquables sur des médias de haute qualité, mais leur efficacité diminue drastiquement lorsque le signal est dégradé par la compression.

Conformément au protocole de l’article [1], j’ai travaillé sur la base de données FaceForensics++ [2], le jeu de référence pour l’évaluation des systèmes de détection de deepfakes. Ce dataset propose trois niveaux de compression : RAW (sans compression), c23 (compression légère) et c40 (compression agressive, simulant les conditions réelles de diffusion sur les réseaux sociaux). J’ai choisi de cibler exclusivement le niveau c40, qui constitue le

scénario le plus difficile et le plus réaliste. C’est précisément pour ces cas pratiques que l’approche fréquentielle justifie son existence : le réseau Xception, référence des méthodes spatiales, voit son Accuracy chuter de 99,26 % sur RAW à 86,86 % sur c40, illustrant concrètement les difficultés des approches spatiales sous compression sévère.

La Figure 1 illustre ce phénomène. Sur les colonnes RAW et HQ, les images générées sont partiellement discernables à l’œil nu. Sur la colonne LQ (c40), le visage manipulé est visuellement indiscernable du visage réel. En revanche, les représentations FAD et LFS révèlent des signatures spectrales distinctes : des anomalies énergétiques localisées sur le visage falsifié dans la colonne FAD, et une distribution statistique des fréquences radicalement différente dans la colonne LFS, notamment au niveau de la région de la bouche.

Sur les données c40, l’architecture F^3 -Net atteint 90,43 % d’Accuracy et une AUC de 0,933, contre respectivement 86,86 % et 0,893 pour Xception seul, soit un gain de 3,57 points sur la métrique principale, comme le confirme la courbe ROC de la Figure 1(c).

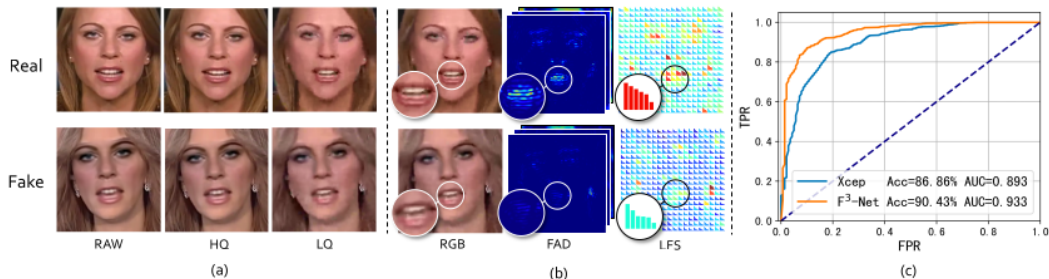


Fig. 1. Frequency-aware tampered clues for face forgery detection. (a) RAW, high quality (HQ) and low quality (LQ) real and fake images with the same identity, manipulation artifacts are barely visible in low quality images. (b) Frequency-aware forgery clues in low quality images using the proposed *Frequency-aware Decomposition (FAD)* and *Local Frequency Statistics (LFS)*. (c) ROC Curve of the proposed **Frequency in Face Forgery Network (F^3 -Net)** and baseline (*i.e.*, Xception [12]). The proposed F^3 -Net wins the Xception with a large margin. Best viewed in color.

FIGURE 1 – Comparaison entre méthode spatiale (Xception) et approche fréquentielle (F^3 -Net) sur données fortement compressées (c40)

2.2 Architecture Globale du Modèle F^3 -Net

Le modèle F^3 -Net repose sur une architecture *two-stream*, c’est-à-dire à deux flux parallèles. L’image d’entrée est dupliquée et traitée simultanément par deux branches spécialisées, chacune conçue pour extraire des informations fréquentielles différentes mais complémentaires. Leurs représentations intermédiaires sont ensuite progressivement fusionnées avant la décision finale (voir Figure 2).

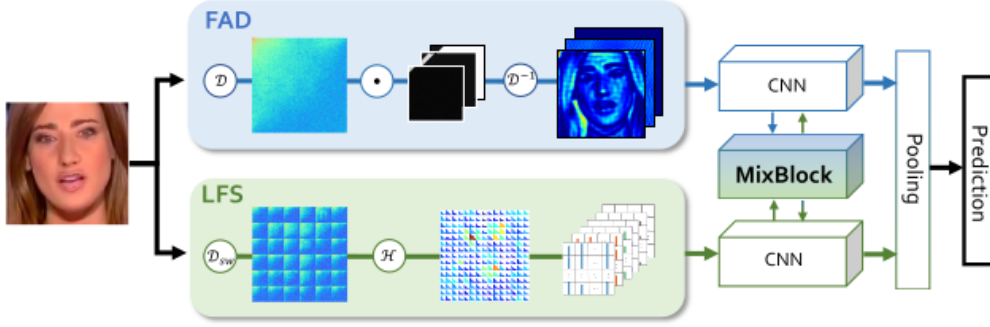


Fig. 2. Overview of the F^3 -Net. The proposed architecture consists of three novel methods: *FAD* for learning subtle manipulation patterns through frequency-aware image decomposition; *LFS* for extracting local frequency statistics and *MixBlock* for collaborative feature interaction.

FIGURE 2 – Architecture globale du F^3 -Net : extraction parallèle par les branches FAD et LFS, extraction de caractéristiques via Xception, et fusion interactive par le MixBlock.

Les trois composants fondamentaux du modèle sont les suivants.

Branche FAD : décomposition spectrale globale. Cette branche prend l’image RGB complète et la décompose selon différentes bandes de fréquences. Concrètement, elle applique une transformée en cosinus discrète (DCT) sur l’image entière, filtre certaines zones du spectre, puis revient dans le domaine spatial. Le résultat est un ensemble de quatre images filtrées, chacune mettant en évidence une plage de fréquences différente. Ces quatre images sont empilées pour former une entrée de 12 canaux (4 bandes $\times$ 3 canaux couleur) transmise au premier réseau Xception.

Branche LFS : statistiques spectrales locales. Cette branche ne regarde pas l’image dans son ensemble mais la découpe en petits blocs de 10×10 pixels via une fenêtre glissante. Pour chaque bloc, elle calcule une DCT locale puis en extrait des statistiques d’énergie sur 6 bandes de fréquences. Le résultat est une carte de taille $149 \times 149 \times 6$ qui encode, pour chaque position spatiale de l’image, la distribution d’énergie spectrale locale un indicateur puissant des zones synthétisées par une IA.

Backbone Xception et MixBlock. Les deux représentations sont injectées dans deux réseaux Xception distincts pré-entraînés sur ImageNet. L’innovation centrale du modèle est le module **MixBlock** : plutôt de fusionner les deux branches uniquement à la fin, ce module les fait communiquer à mi-parcours (après les blocs 7 et 12 de Xception). Si la branche FAD détecte une anomalie globale dans une région, elle le signale à la branche LFS qui peut alors y concentrer son analyse locale, et inversement. Après le bloc 12, les

deux représentations finales sont concaténées en un vecteur de dimension 4096, suivi d'un dropout et d'une couche linéaire pour la classification binaire réel/falsifié.

2.3 Décomposition Adaptative Globale (FAD)

Principe général

Le module FAD répond à une question simple : comment filtrer une image pour ne garder que les fréquences où se cachent les artefacts de falsification ? L'approche naïve serait d'appliquer un filtre passe-haut fixe. La contribution de l'article [1] est de rendre ce filtrage **adaptatif** : les frontières entre les bandes de fréquences sont apprises pendant l'entraînement, plutôt que fixées à la main.

Formulation mathématique du filtrage

La DCT appliquée à une image la transforme en une carte de coefficients de fréquences : les basses fréquences (variations lentes, structure globale) se retrouvent dans le coin supérieur gauche, et les hautes fréquences (détails fins, textures) dans le coin inférieur droit. Les artefacts de falsification laissent des traces dans certaines zones de cette carte.

L'article définit l'opération de filtrage pour un canal c de l'image x et une bande i par :

$$y_i = \mathcal{D}^{-1} \left\{ \mathcal{D}(x) \odot [f_{base}^i + \sigma(f_w^i)] \right\} \quad (1)$$

où $\mathcal{D}$ désigne la DCT bidimensionnelle, $\mathcal{D}^{-1}$ son inverse (IDCT), et $\odot$ le produit de Hadamard.

Le filtre $[f_{base}^i + \sigma(f_w^i)]$ se compose de deux termes :

- f_{base}^i est un masque binaire fixe, qui délimite grossièrement la bande de fréquences i dans la carte DCT. L'article utilise 3 bandes fixes (basses, moyennes, hautes fréquences). Conformément à l'implémentation du dépôt [4], mon code intègre un filtre supplémentaire (*all-pass*) qui conserve l'image originale non filtrée, fournissant ainsi au réseau un point de référence global. En pratique, on a donc $N=4$ bandes.
- f_w^i est une matrice de paramètres appris par le réseau, qui affine les frontières du filtre de base. La fonction σ le maintient dans l'intervalle $] -1, 1[$, ce qui garantit que le filtre appris ne peut que nuancer le masque fixe, sans le contredire complètement :

$$\sigma(f_w^i) = \frac{1 - \exp(-f_w^i)}{1 + \exp(-f_w^i)} \in] -1, 1[\quad (2)$$

En pratique, pour chaque bande i et chaque canal couleur c , on obtient une image filtrée $y_{i,c}$. La concaténation des 4 bandes sur les 3 canaux produit un tenseur de 12 canaux transmis au réseau Xception.

Mes choix d'implémentation

La DCT est implémentée comme une multiplication matricielle $\mathcal{D}(x_c) = \mathcal{D}_H \cdot x_c \cdot \mathcal{D}_H^\top$, ce qui la rend différentiable et directement intégrable dans PyTorch. J'ai repris ce choix de [3], qui définit la matrice $\mathcal{D}_H$ par ses coefficients cosinus standards.

La première couche convolutive du backbone Xception, initialement conçue pour 3 canaux RGB, a été adaptée pour accepter ces 12 canaux. Conformément à l'implémentation de [3], j'ai redistribué les poids pré-entraînés sur ImageNet en les copiant quatre fois (une par bande) et en les divisant par 4.

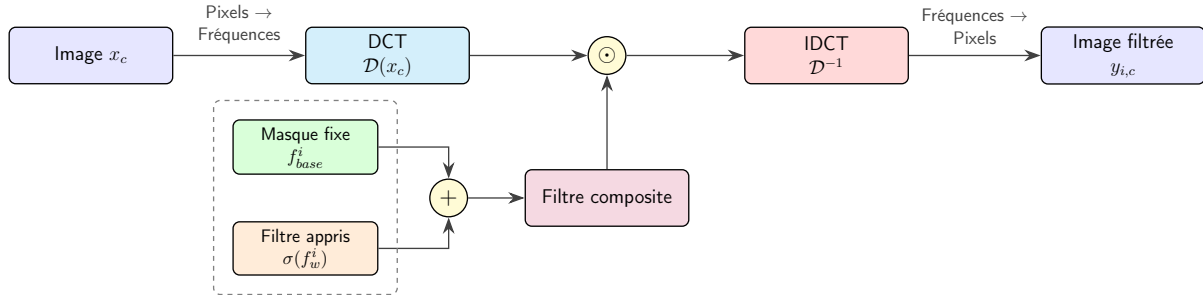


FIGURE 3 – Pipeline du module FAD. L'image est projetée dans le domaine fréquentiel par DCT, filtrée par un masque adaptatif, puis reprojétée dans le domaine spatial. L'opération est répétée pour 4 bandes $\times$ 3 canaux.

2.4 Statistiques de Fréquences Locales (LFS)

Principe général

Le module LFS part d'un constat différent : les artefacts de falsification ne sont pas distribués uniformément sur tout le spectre global, mais se manifestent sous forme d'incohérences locales. Une zone de peau synthétisée par une IA a une texture microscopique différente d'une peau réelle, même si globalement l'image semble normale. Pour détecter cela, le module découpe l'image en petits blocs et analyse les fréquences de chaque bloc indépendamment.

Formulation mathématique de l'agrégation locale

L'article [1] définit la statistique de la bande i pour un patch local p par :

$$q_i = \log_{10} \left\| \mathcal{D}(p) \odot [h_{base}^i + \sigma(h_w^i)] \right\|_1 \quad (3)$$

Détaillons chaque terme :

- $\mathcal{D}(p)$ est la DCT appliquée au patch local p . Contrairement au FAD qui applique la DCT à l'image entière, ici on calcule une DCT par bloc, d'où le nom SWDCT (*Sliding Window DCT*).

- $[h_{base}^i + \sigma(h_w^i)]$ est le même type de filtre adaptatif que dans FAD, mais appliqué aux $M = 6$ bandes de LFS. Il isole la contribution de la bande i dans le spectre du patch.
- $\log_{10}$ permet de limiter l'influence des écarts d'amplitude entre les basses et les hautes fréquences.

Pour une image de 299×299 pixels, avec une fenêtre de taille 10 et un pas de 2, on obtient $149 \times 149 = 22\,201$ patches. En calculant le vecteur $(q_1, \dots, q_6)$ pour chacun et en les réassemblant, on reconstruit une carte de taille $149 \times 149 \times 6$, qui encode pour chaque position la distribution d'énergie spectrale locale.

Mes choix d'implémentation

L'image est d'abord convertie en niveaux de gris par pondération photométrique standard $(0,299 R + 0,587 G + 0,114 B)$, conformément à [3], puis remise à l'échelle dans $[0, 255]$. Le découpage en blocs est réalisé par un opérateur `Unfold` de PyTorch, ce qui évite une boucle explicite et reste différentiable. Ces choix d'implémentation sont identiques à ceux de [3].

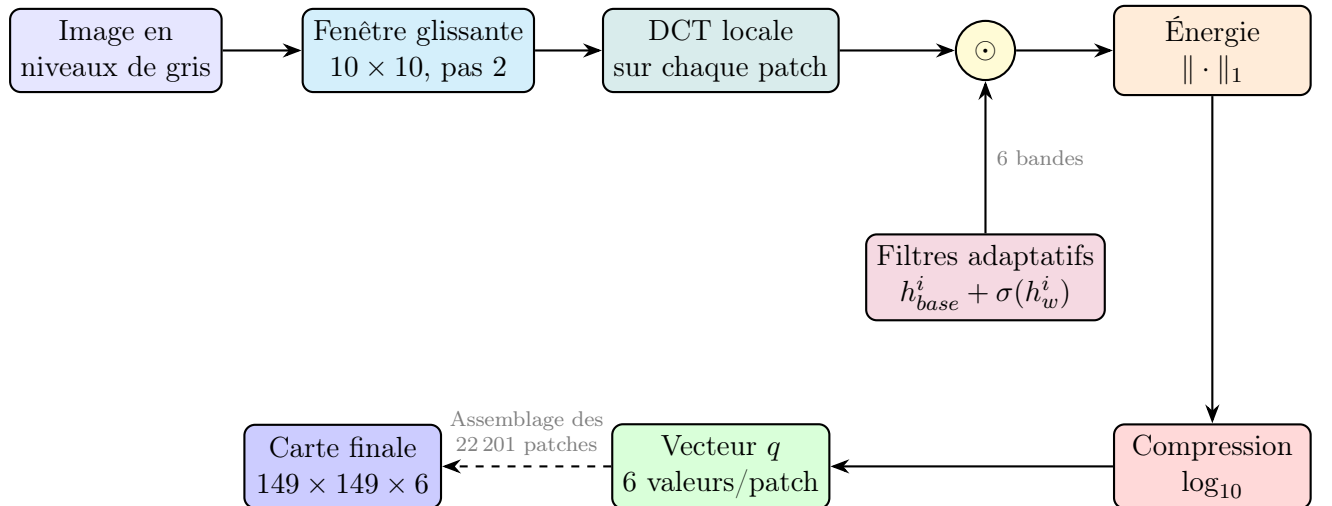


FIGURE 4 – Pipeline du module LFS. L'image est découpée en blocs, chaque bloc est analysé spectralement sur 6 bandes, et les statistiques sont réassemblées en une carte spatiale.

2.5 Fusion par Attention Croisée (MixBlock)

Principe général

À ce stade, les deux branches produisent des représentations complémentaires : FAD encode les anomalies globales du spectre, LFS encode les incohérences locales de texture. L'article [1] propose de ne pas simplement concaténer ces représentations en fin de réseau,

mais de les faire communiquer à mi-parcours. L'intuition est simple : si FAD détecte une zone suspecte, LFS devrait y prêter plus d'attention, et réciproquement.

Mécanisme d'attention croisée

Le MixBlock est appliqué deux fois : après le bloc 7 et après le bloc 12 du backbone Xception. À chaque application, il prend en entrée les cartes de caractéristiques des deux branches **A** (FAD) et **B** (LFS), et calcule une matrice d'attention Ω qui encode les correspondances spatiales entre les deux flux.

Cette matrice est ensuite utilisée pour enrichir chaque branche avec l'information de l'autre, via une mise à jour résiduelle :

$$\tilde{\mathbf{A}} = \mathbf{A} + \text{BN}(\psi_A(\mathbf{B} \odot \Omega \odot (2\sigma(\gamma_B) - 1))) \quad (4)$$

$$\tilde{\mathbf{B}} = \mathbf{B} + \text{BN}(\psi_B(\mathbf{A} \odot \Omega \odot (2\sigma(\gamma_A) - 1))) \quad (5)$$

où ψ_A et ψ_B sont des convolutions 1×1 , BN une normalisation par batch, et γ_A, γ_B des scalaires appris initialisés à 0. Ce dernier point est important : au début de l'entraînement, $2\sigma(0) - 1 = 0$, donc le MixBlock ne fait rien, les branches apprennent d'abord indépendamment, et le couplage s'active progressivement au fil de l'entraînement.

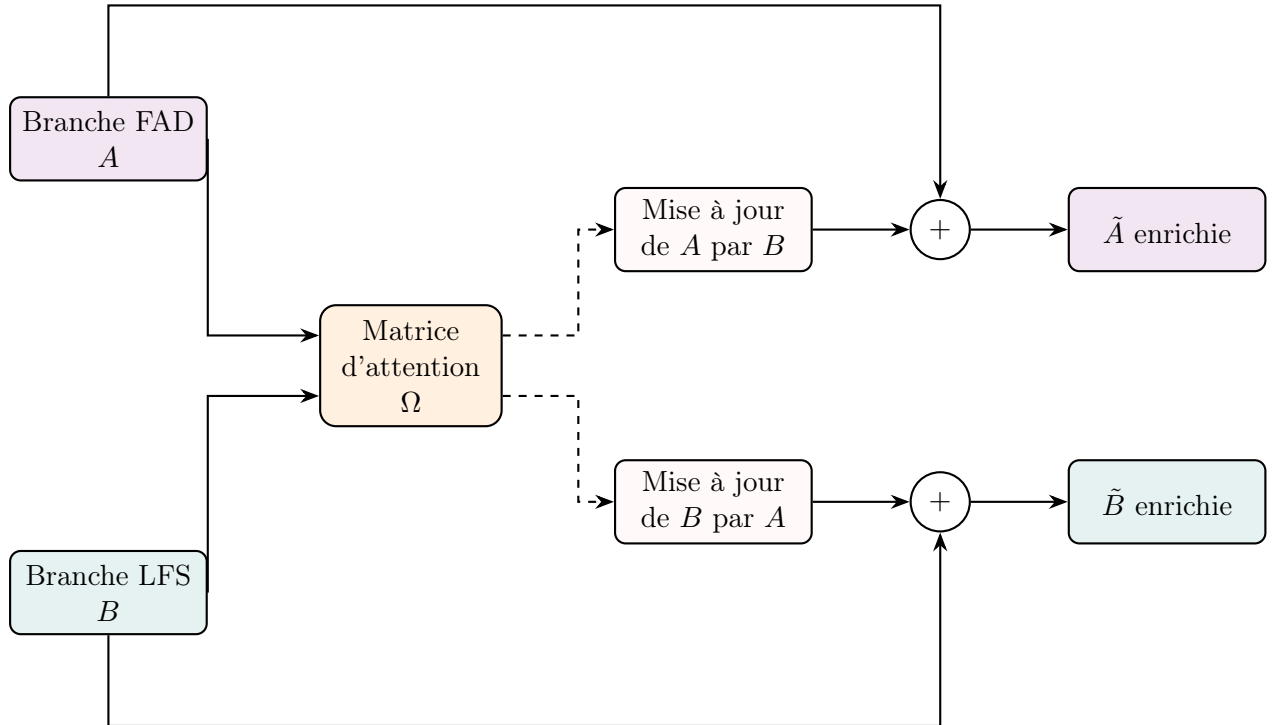


FIGURE 5 – Module MixBlock. Les deux branches calculent conjointement une matrice d'attention Ω , qui permet à chaque branche d'être enrichie par les caractéristiques de l'autre via une connexion résiduelle (lignes pleines en haut et en bas) qui contourne le bloc de mise à jour central.

3 Méthodologie et Pipeline Logiciel

3.1 Le Dataset FaceForensics++ (FF++)

FaceForensics++ [2] est constitué de 1 000 vidéos authentiques collectées sur YouTube, chacune manipulée par quatre méthodes DeepFakes, Face2Face, FaceSwap et NeuralTextures, pour produire 5 000 vidéos au total. Le protocole officiel partitionne le jeu en 720 vidéos d’entraînement, 140 de validation et 140 de test, réparties uniformément entre vidéos réelles et manipulées.

Les quatre méthodes de falsification. DeepFakes et FaceSwap procèdent à un échange complet du visage, introduisant des artefacts visibles aux frontières du nouveau visage. Face2Face réencode les expressions faciales d’une source vers une cible, produisant des incohérences de texture plus difficiles à détecter. NeuralTextures est la méthode la plus difficile à détecter : elle ne modifie que la région de la bouche via un rendu neural, laissant des traces quasi-imperceptibles. Cette hiérarchie de difficulté se retrouve dans les résultats : sur c40, F^3 -Net atteint 97,97 % sur DeepFakes mais seulement 83,32 % sur NeuralTextures.

Équilibrage des classes. Le dataset présente un déséquilibre naturel : il y a quatre fois plus de vidéos falsifiées que de vidéos réelles (4 000 contre 1 000). Pour compenser, le chargeur de données de [3] rééchantillonne séparément les vidéos réelles et falsifiées, en construisant chaque batch avec un ratio 1 :1. Pour chaque vidéo, 270 frames sont échantillonnées selon le protocole de l’article, ce qui produit un total de 1 349 615 patches de visage après prétraitement.

3.2 Prétraitement : Justification du Crop Conservateur

Les réseaux convolutifs opèrent sur des images de taille fixe (299×299 pixels pour Xception). J’ai développé un script de prétraitement dédié pour extraire les régions faciales à partir des vidéos brutes, ni l’article ni les dépôts disponibles ne le fournissent. Appliqué à 5 000 vidéos, ce pipeline a produit 1 349 615 patches de visage en environ 40 heures de traitement sur GPU.

Détection du visage par MTCNN. Pour chaque vidéo, 270 frames sont échantillonnées à intervalles réguliers selon le protocole de l’article [1]. Sur chacune, le détecteur MTCNN localise le visage le plus visible et retourne une boîte englobante (x_1, y_1, x_2, y_2) .

Crop conservateur à facteur $\times 1,3$. Conformément aux spécifications de l’article [1], la boîte englobante est agrandie d’un facteur 1,3 autour de son centre :

$$x'_1 = c_x - \frac{1,3 \cdot (x_2 - x_1)}{2}, \quad x'_2 = c_x + \frac{1,3 \cdot (x_2 - x_1)}{2} \quad (6)$$

Ce facteur est motivé par la nature même des algorithmes de falsification : les méthodes d'échange de visage opèrent à la frontière du visage pour rendre réaliste le recollement de ce nouveau visage. Cette zone de transition, absente d'un crop serré mais capturée par le facteur $\times 1,3$, contient précisément les artefacts spectraux les plus discriminants pour le module FAD. La région est ensuite redimensionnée en 299×299 pixels par interpolation de Lanczos.

Les patches extraits ont ensuite été répartis dans des répertoires distincts (train, valid, test) et séparés par classes (réel / falsifié) afin d'être directement exploitables par le chargeur de données PyTorch du dépôt de référence [3].

3.3 Architecture Logicielle et Choix Techniques

Les deux dépôts de référence : Deux implémentations ont guidé ce travail. Le dépôt [3] fournit l'architecture complète du modèle : modules FAD, LFS et MixBlock intégré dans le *forward pass* aux blocs 7 et 12, avec une couche de classification $4096 \rightarrow 2$. Il s'agit d'un dépôt centré sur le modèle : il ne contient ni boucle d'entraînement, ni chargeur de données, ni gestion des métriques. Le dépôt [4] fournit quant à lui un pipeline d'entraînement complet (`train.py`, `trainer.py`, `utils.py`) avec le chargeur `FFDataset` adapté à la structure de FF++, mais son implémentation du modèle ne connecte pas le MixBlock dans la boucle d'inférence alors que je considère qu'il s'agit pourtant de la contribution centrale de l'article, censée permettre la communication entre les deux branches à mi-parcours plutôt qu'une simple fusion finale.

Stratégie de combinaison. Ne pas disposer du MixBlock revenait à se priver de l'apport principal de l'architecture F^3 -Net. J'ai donc opté pour une combinaison des deux dépôts : l'architecture de [3] comme modèle cible, et le pipeline de [4] comme infrastructure d'exécution. Cette combinaison a nécessité plusieurs adaptations :

- `f3net.py` de [3] remplace `models.py` dans le pipeline de [4] : c'est le changement principal, qui garantit que le MixBlock est bien actif pendant l'entraînement.
- J'ai modifié `trainer.py` pour passer le chemin du checkpoint Xception en argument, supprimant ainsi le chemin relatif codé en dur dans [3].
- J'ai remplacé la `BCEWithLogitsLoss` de [4] par une `CrossEntropyLoss` : la première est conçue pour une sortie scalaire (classification binaire à 1 neurone), la seconde est adaptée à une sortie de dimension 2 avec des labels entiers, conformément à l'architecture de [3].

3.4 Remarques sur la Reproductibilité

Absence de script de prétraitement. Ni l'article, ni les deux dépôts ne fournissent de pipeline d'extraction de frames ni de détection faciale. Ce script a dû être développé intégralement depuis les descriptions de l'article : 270 frames par vidéo, facteur $\times 1,3$ autour du centre du visage détecté, redimensionnement en 299×299 pixels par interpolation de Lanczos.

Divergence entre les dépôts et l'article original. En lisant attentivement les deux dépôts, j'ai remarqué quelques divergences avec le protocole décrit dans l'article.

D'abord, pour compenser le déséquilibre du dataset (4 fois plus de vidéos falsifiées que de vidéos réelles), le pipeline hérité du dépôt [4] construit chaque batch à partir de deux chargeurs de données échantillonnés séparément, garantissant un ratio 1 :1 entre échantillons réels et falsifiés. Cette stratégie diffère de celle décrite dans l'article [1], qui suréchantillonne les vidéos réelles par un facteur 4 en amont du chargement plutôt que d'imposer un ratio strict à chaque batch. J'ai choisi de conserver le choix du dépôt [4] car il apporte une solution similaire.

Un autre changement concerne la méthodologie d'entraînement. L'article [1] entraîne F^3 -Net avec SGD ($\text{lr} = 0,002$, momentum 0,9) couplé à un scheduler cosine annealing avec un batch size de 128. Le dépôt [4] s'écarte de ce protocole et utilise Adam à la place, avec un batch size de 12. Par curiosité, j'ai décidé de tester les deux approches pour comprendre et documenter ce choix. Par contre, j'ai été contraint d'utiliser un batch size de 12, contrainte directement liée à la VRAM des GPU disponibles sur le cluster de Télécom Paris.

4 Expérimentations et Analyse des Résultats

4.1 Phase d'Entraînement

J'ai entraîné et comparé deux configurations sur le protocole c40 de FaceForensics++, toutes limitées à 5 epochs compte tenu des ressources allouées, avec une sauvegarde du meilleur modèle (*checkpoint*) basée sur l'AUC du jeu de validation.

v1 — Adam, batch 12. Configuration de référence, conforme aux dépôts [3] et [4] : optimiseur Adam ($\text{lr} = 0,0002$, $\beta_1 = 0,9$, $\beta_2 = 0,999$), batch size de 12.

v2 — SGD + cosine annealing, batch 12. Reproduction du protocole de l'article [1] : SGD ($\text{lr} = 0,002$, momentum 0,9) avec scheduler cosine annealing.

4.2 Performances et Analyse Critique

Conformément à l'appendice 6.1 de l'article [1], qui décrit une recherche empirique du seuil de décision optimal (θ) sur la validation pour maximiser l'Accuracy, j'ai implémenté cette même procédure de calibration post-entraînement :

La méthode algorithmique est la suivante :

1. **Inférence sur la validation :** Le modèle évalue l'intégralité du jeu de validation et retourne un score de probabilité brut (via la couche *Softmax*) pour chaque image.
2. **Recherche empirique :** Un balayage de l'espace des seuils (de 0 à 1 avec un pas de 0,01) est effectué sur la courbe ROC du jeu de validation afin d'identifier le seuil optimal θ qui maximise mathématiquement l'Exactitude globale (*Accuracy*).
3. **Évaluation sur le test :** Ce seuil θ calibré est ensuite appliqué de manière statique au jeu de test (données jamais vues par le modèle) pour évaluer les performances

finale.

Cette approche a permis de rééquilibrer le jugement du réseau avant d’extraire les métriques présentées dans le Tableau 1.

Configuration	Test AUC	Acc (seuil 0,5)	Acc (seuil opt.)	Seuil θ	r_acc	f_acc
Xception (baseline article [1])	0,8930	—	—	—	—	—
F^3 -Net (article [1])	0,9330	—	—	—	—	—
v1 — Adam, batch 12	0,8778	0,8422	0,8476	0,06	0,4607	0,9443
v2 — SGD + cosine, batch 12	0,8658	0,8183	0,8470	0,07	0,4881	0,9367

TABLE 1 – Comparaison détaillée des performances sur le jeu de test c40. L’optimisation du seuil de décision (θ) permet de mitiger systématiquement le biais de prédiction du modèle. (r_acc : vidéos réelles, f_acc : vidéos falsifiées).

Comparaison (v1 vs v2). La configuration v1 (Adam) obtient la meilleure AUC (0,8778), validant empiriquement le choix d’implémentation des dépôts [3, 4]. Dans ce régime d’entraînement contraint (batch size de 12, 5 époques), l’optimiseur SGD (v2) dispose d’un signal de gradient trop bruité pour converger aussi efficacement qu’un optimiseur adaptatif. L’architecture originale [1], entraînée sur des batches de 128, bénéficie d’une estimation de la moyenne des gradients beaucoup plus stable qui fait cruellement défaut sur des infrastructures limitées. On note toutefois que la configuration SGD parvient à une représentation légèrement plus équilibrée des visages réels (r_acc de 0,4881 contre 0,4607).

Biais de classe et ajustement du seuil de décision (θ). Une observation majeure réside dans le déséquilibre entre la *Fake Accuracy* ($> 0,93$) et la *Real Accuracy* ($< 0,50$). Le réseau développe un biais conservateur extrêmement prononcé : face à la difficulté mathématique de garantir l’absence totale d’artefacts sur un vrai visage (surtout après une compression destructrice c40), il tend à prédire la classe falsifiée par défaut au moindre doute. Pour mitiger ce phénomène, j’ai calculé le seuil de décision optimal θ sur la courbe ROC de validation. L’application de ces seuils très bas (compris entre 0,06 et 0,07) sur le jeu de test force mécaniquement le modèle à rééquilibrer sa confiance, ce qui a permis d’améliorer systématiquement l’*Accuracy* globale du modèle final.

5 Conclusion

Ce projet intégrateur s’est révélé être bien plus qu’un simple exercice de reproduction. Implémenter l’architecture F^3 -Net m’a permis de mesurer pour la première fois l’écart qui sépare la théorie parfaite d’un article de recherche de la réalité du développement dans un environnement contraint.

Même si ce rapport laisse de côté les erreurs rencontrées, les moments de débogage et les diagnostics qu’il a fallu poser, c’est paradoxalement la partie du travail que j’ai préférée. C’est précisément dans ces moments de blocage que j’ai le plus appris.

Je ressors donc de ce projet avec des compétences techniques renforcées, mais surtout avec l'envie de continuer à mettre en application l'intelligence artificielle pour résoudre des problématiques concrètes.

Références

- [1] Qian, Y., Yin, G., Sheng, L., Chen, Z., & Shao, J. (2020). *Thinking in Frequency : Face Forgery Detection by Mining Frequency-aware Clues*. European Conference on Computer Vision (ECCV).
- [2] Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). *FaceForensics++ : Learning to Detect Manipulated Facial Images*. Proceedings of the IEEE International Conference on Computer Vision (ICCV).
- [3] Leminhbinh0209 (2021). *Thinking in frequency : Face forgery detection by mining frequency-aware clues — Reference implementation*. <https://github.com/Leminhbinh0209/F3Net>
- [4] yyk-wew (2021). *Implementation of F3-Net : Frequency in Face Forgery Network*. <https://github.com/yyk-wew/F3Net>